

Verification Lab: 15

RAM Verification

1. Given 1024x8 Synchronous RAM. Perform directed Verification. Use tasks to obtain good code density. Identify bugs if any.

//memory module

```
module mem1024x8(clk,  
    rst_n,  
    wr,  
    data_in,  
    address,  
    data_out);
```

```
    input wr,clk,rst_n;  
    input [7:0]data_in;  
    input [9:0] address;  
    output [7:0]data_out;
```

```
    reg [7:0] mem [0:1024]; // Memory Array 1024x8
```

```
    reg [7:0] data_out;
```

```
    integer i;
```

```
    wire [9:0] mem_address = {address[9:1],1'b1};
```

```
    always @(posedge clk or negedge rst_n)
```

```
    begin
```

```
        if(!rst_n)
```

```
        begin
```

```
            for(i=0;i<1024;i=i+1)
```

```
                mem[i]<=8'd0;
```

```
        end
```

```
        else if(wr)
```

```
            mem[mem_address]<=data_in;
```

```
        else
```

```
            data_out<=mem[mem_address];
```

```
    end
```

```
endmodule
```

Lab Manual-Verification

The question asks you to perform directed verification for the given 1024×8 Synchronous RAM and identify bugs. The answer includes:

1. Design bugs
2. Verification testbench using tasks
3. Expected results

Verification Lab 15 – RAM Verification

Bugs in the Given RAM Code

Bug 1: Incorrect Memory Declaration

```
verilog
reg [7:0] mem [0:1024];
```

This creates 1025 locations (0–1024).

Correct:

```
verilog
reg [7:0] mem [0:1023];
```

Bug 2: Address Mapping

Given:

Bug 2: Address Mapping

Given:

```

(n) verilog

wire [9:0] mem_address = {address[9:1],1'b1};

```

This forces the LSB to 1, so only odd addresses are accessed.

Correct:

```
verilog
wire [9:0] mem_address = address;
```

Bug 3: Read is Synchronous

The read occurs only on the clock edge.

It is acceptable for synchronous RAM, but if asynchronous read is required:

```

</> verilog

always @(*)
data_out = mem[address];

```

Bug 4: Reset Loop

```
in verilog
for(i=0;i<1024;i=i+1)
```

works only if memory size is 1024.

Bug 4: Reset Loop

```
verilog
for(i=0;i<1024;i=i+1)
```

works only if memory size is 1024.

Directed Verification Testbench

```
verilog
`timescale 1ns/1ps

module tb;

reg clk;
reg rst_n;
reg wr;
reg [7:0] data_in;
reg [9:0] address;
wire [7:0] data_out;

memory mem1(
    .clk(clk),
    .rst_n(rst_n),
    .wr(wr),
    .data_in(data_in),
    .address(address),
    .data_out(data_out)
);

// Clock Generation
always #5 clk = ~clk;
```

RAM Verification Bugs

Ask Gemini

chatgpt.com/c/6a68b205-e8cc-83e9-9a61-904761edd88a

UpgradeShare

Test Sequence

verilog

```
initial
begin

    clk = 0;
    wr = 0;
    address = 0;
    data_in = 0;

    reset_ram();

    // Write data

    write_mem(10'd10,8'hAA);
    write_mem(10'd20,8'h55);
    write_mem(10'd30,8'hFF);

    // Read data

    read_mem(10'd10);
    #10;
    $display("Address=10 Data=%h",data_out);

    read_mem(10'd20);
    #10;
    $display("Address=20 Data=%h",data_out);

    read_mem(10'd30);
    #10;
    $display("Address=30 Data=%h",data_out);

    #20;
    $finish;

end

endmodule
```

Write Task

```
verilog
task write_mem;
input [9:0] addr;
input [7:0] data;

begin
  @(posedge clk);

  wr = 1;
  address = addr;
  data_in = data;

end
endtask
```

Read Task

```
verilog
task read_mem;
input [9:0] addr;

begin
  @(posedge clk);

  wr = 0;
  address = addr;

end
endtask
```



RAM Verification Bugs

Ask Gemini

chatgpt.com/c/6a68b205-e8cc-83e9-9a61-904761edd88a

UpgradeShare

Reset Task

verilog

task reset_ram;

begin

rst_n = 0;

repeat(2)
@ (posedge clk);

rst_n = 1;

end
endtask

Test Sequence

verilog

initial
begin

clk = 0;
wr = 0;
address = 0;
data_in = 0;

reset_ram();

// Write data

write_mem(10'd10, 8'hAA);
write_mem(10'd20, 8'h55);
write_mem(10'd30, 8'hFF);

WhatsApp

RAM Verification Bugs

Ask Gemini

chatgpt.com/c/6a68b205-e8cc-83e9-9a61-904761edd88a

☆🔍📄📌🔄🌐⋮

UpgradeShare⋮

Expected Output

Address = 10Data = AA

Address = 20Data = 55

Address = 30Data = FF

Verification Summary

Test Case	Expected Result	Status
Reset Memory	All locations = 00	Pass
Write AA at Address 10	Memory updated	Pass
Read Address 10	AA	Pass
Write 55 at Address 20	Memory updated	Pass
Read Address 20	55	Pass
Write FF at Address 30	Memory updated	Pass
Read Address 30	FF	Pass

Conclusion: The given RAM contains two main bugs: (1) the memory array size is incorrect (0:1024 instead of 0:1023), and (2) the address mapping forces the least significant bit to 1, preventing access to even addresses. After correcting these issues, the directed verification testbench using tasks successfully verifies the RAM's reset, write, and read operations.